Changelog & Initial Approval

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.220Z

mattock_name / real_coder

Project ID: 697b72cae052640b8db3e22d

Changelog

Change Log (only QC makes changes to the doc after initial version approved)						
Date	**Time**	**Summary of changes**	**Customer Centered reason for change**	**Changes requested by (link to reference)**	**Requires MLDG Input?**	**MLDG Approved by *(Signoff Change)***

Mar 25, 2026	**11:19 AM CT**	*Search **03/25***
		- *Added:*
		  - *Golden Response - Failing Verifiers subdimension*
		  - *[Fail - Golden Response failing verifier]*
		  - *Checks if the golden response fails any rubric criteria/unit tests*
		- *Changed*
		  - *Test Suite - Overly Specific subdimension*
		  - *Now Overly Specific/Broad*
		  - *[Fail - More Than 5% Overly Specific Tests] changed to [Fail - More Than 10% Overly Specific/Broad Tests]*
		  - *Overly broad tests are now also considered for this error–broad/specific tests together should be no more than 10% of tests.*
				Onsang Yau	Yes	

Mar 23, 2026	**2:10 PM PST**	- *"Minor Issues > Miscategorized Criteria" subdimension: Added a note saying if multiple rubric categories are applicable, any of them can be selected without getting penalized*			Yes	
Mar 21, 2026	**8:56 AM CT**	*Search **03/21*** - *"Prompt > Feasibility"* - *Added note regarding possible Markdown rendering issues*			Yes	

Mar 17, 2026	**5:45 PM PST**	● *Prompt > Feasibility: Added a note saying that prompts's requests can be internet-dependent now, but still not require external APIs*
		● *Verifiers > Verifier Coverage:*
		  ○ *Added a note and an example for when coverage for a missing test is accounted for by other tests collectively*
		  ○ *[Fail - Major Insufficient Verifier Coverage]: Added a 5% room for missing verifier test coverage*
					Yes	

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.220Z

Mar 16, 2026	**5 PM PST**	• *"Golden Response > Execution Output"*
		  ○ *Added an exception for insignificant issues with minor edge cases to be non-failing*
		  ○ *Moved runtime errors from "Golden Response > Compilation" into this dimension*
					Yes	

Mar 12, 2026		• *"Prompt > Expected Interfaces":*
		  ○ *Added [Fail - Invalid Interface]: Internal helper functions or third party fields that are not necessary for external applications to work should not be documented anymore*
		• *"Test Suite > Overly Specific"*
		  ○ *Added a note to reference the examples under "Overfitting and Underfitting" rubric dimension*
		• *"Overfitting and Underfitting" rubric dimension*
		  ○ *Reworded the criteria to be clearer and added more examples*
					Yes	

Mar 7, 2026	**4:45 PM PST**	*Added clarifications to the Overfitting/Underfitting rubric*			Yes	
Mar 4, 2026	**5:40 PM CT**	*Search 03/04:* - *Added a note to the "Verifier Coverage" dimension that excludes subjective and minor UI design requirements*			Yes	
Feb 27, 2026	**3:45 PM PST**	• *Added a note to "Verifiers > Verifier Coverage" regarding "bottleneck" tests*			Yes	
Feb 26, 2026	**9:15 AM PST**	• *"Criteria Not Atomic - Major": Expanded the notes saying that features that are closely related can also be grouped together into one criterion*			Yes	
Feb 24, 2026	**3:15 PM PST**	• *Added "Test Suite > Overly Specific" rubric dimension for flagging overfitted unit tests*			Yes	
Feb 24, 2026	**1:30 PM PST**	• *Slightly modified the notes for "Expected Interface(s)" dimension*			Yes	

Feb 22, 2026	**11:55 PM CT**	*Search 2/22*
		- *Added notes to `Missing Criteria - Critical Requirements` regarding:*
		  - *>3 missing important explicit criteria*
		  - *how it applies for tasks with <30 explicit request criteria*
		  - *how it applies for tasks with >=30 explicit request criteria*
		- *Added clarifying note to Expected Interface(s) regarding [Fail - Undocumented Interface] that explains that helper functions and trivial imports do not need to be added as interfaces and it should only be publicly accessible to external applications or something the test suite will interact with*
		- *Removed `Code Documentation` section entirely*
					Yes	

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.220Z

		- *Added notes to `Verifier Coverage` that explain:*
		  - *the test suite is meant to follow the rewritten prompt not the golden patch*
		  - *it's okay for tests to be generic to account for multiple valid solutions to the prompt*
		  - *To explain [Fail - Major Insufficient Verifier Coverage] is when the test suite doesn't cover a major backed requirement of the prompt and the rubric doesn't cover it at all*				

Feb 22, 2026	**05:30 PM CT**	*Search* ***02/22***
		- *"Verifiers - Verifier Coverage"*
		  - *Updated to account for new "Missing Criteria" rubric errors (there was overlap before)*
		  - *Updated "Fail - Major Insufficient Verifier Coverage" to fail if the test suite does not have at least 80% coverage*
					Yes	

Feb 21, 2026	**Noon PST**	*Clarified Missing Criteria coverage. (Requirements loosen after 30 critical criteria get covered)*			Yes	

Feb 20, 2026	**5:40 PM CT**	*Search* ***02/20:***
		- *Added*
		  - *Workflow Step 4: F2P tests*
		  - *Note on running the validation script to check F2P tests.*
		- *Updated*
		  - *Criteria Atomic - Major*
		- *Removed*
		  - *Criteria Atomic - Minor*
					No	

Feb 19, 2026	**12:00 PM PST**	*Search* ***02/19:***
		● *Ported rubric evaluation criteria to a holistic framework where all the errors across the rubric are counted towards shared Major / Moderate / Minor error thresholds*
		  ○ *Added "Overall Rubric Quality" dimension*
		  ○ *Added "Rubric Quality Definitions" table in the appendix*
		● *Data Specification Completeness: Removed build script from the list of critical fields*
					Yes	

Feb 18, 2026	**6:15 PM CT**	*Search* ***02/18:***
		- ~~*Changed*~~
		  - ~~*Thresholds changed for all rubric criteria errors:*~~
		  - ~~*5% Threshold Errors*~~
		  - ~~*Accuracy*~~
		  - ~~*Missing Coverage*~~
		  - ~~*Atomicity*~~
		  - ~~*Objectively Wrong*~~
				Jean Xin *Onsang Yau*	No	

com+outlier | 2026-04-20T17:55:21.220Z
CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.220Z

		~~- Counterproductive~~ ~~- 10% Threshold Errors~~ ~~- Overlap/Redundancy~~ ~~- Irrelevant Criteria~~ ~~- 20% Threshold Errors~~ ~~- Labels/Annotations~~ *The above changes do not matter anymore as we moved to a different evaluation framework.*				
Feb 18, 2026	**7:30 AM PST**	*- Updated dimensions to reflect the update (5 instead of 4, Visual replaced with Code Clarity & Code Efficiency)* *- Clarified that Rubrics coverage need not include items already being tested by unit tests*			No	
Feb 4, 2026	**08:31 AM PST**	*Search* **02/04** *- "Rubric Criteria - Coverage"* *- Added additional note regarding non-critical explicit instructions* *- Updated "Fail - Missing Coverage"* *- Tasks should only fail if the missing criterion should be weighed as "Mandatory"* *- Updated "Non-Fail - Missing Coverage"* *- Missing non-critical explicit instructions can be penalized as non-failing issues*			No	
Feb 2, 2026 11:38 AM	**11:38 pm CT**	*Search 02/02:* *Added:* *- Verifier Coverage subdimension* *- Note regarding optional instructions & distinction between optional instructions and mandatory instructions for features that have an optional component in a different context.* *- Prompt - Expected Interface(s) new subdimension* *- Rewritten prompts must now include an "Expected Interfaces" section which documents publicly accessible/verifier-tested components of the solution. See subdimension notes for required fields & descriptions thereof.* *- New Errors:* *- [Fail - Missing Interface Section] for omission of the section or any required field within.* *- [Fail - Undocumented Interface] for an added publicly accessible/tested-by-verifier component that is undocumented* *- [Fail - Misleading Interface Description] for interface descriptions which are incomplete/insufficient such that a*			No	

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.220Z

verifier would fail a response which completely fulfilled the requirements listed.		

Doc Initial Approvals

	Named DRI	Signoff (add date)
EM	*Jean Xin*	**Jan 30, 2…**
STO		
QC DRI		

MLDG DRI	

<no MLDG doc signoff required for initial doc, only grading rubrics>

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.220Z

Grading Instructions

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.220Z

mattock_name / real_coder

Project ID: 697b72cae052640b8db3e22d

Project Context:

Objective: The goal of this project is the creation of high-quality, verified software solutions for freelance-style tasks starting from a blank slate. Auditors must evaluate three distinct artifacts provided by the contributor:

1. The Agent Prompt: A structured entry point designed to guide an AI agent in solving the task from scratch.
2. The Golden Patch: A fully functional, high-quality "ground truth" implementation.
3. The Dual-Layer Verification Suite: Automated F2P (Fail-to-Pass) test cases
4. Multi-dimensional expert rubric: based on the prompt requirements, contributors create a tailored rubric to rate the golden patch.

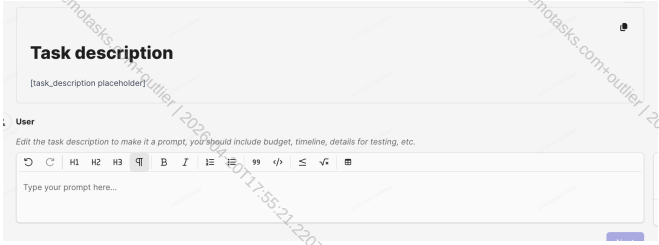
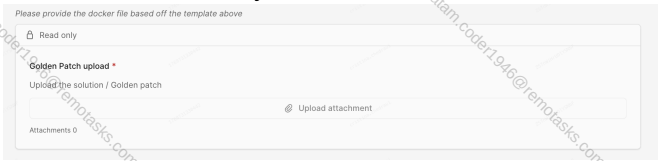
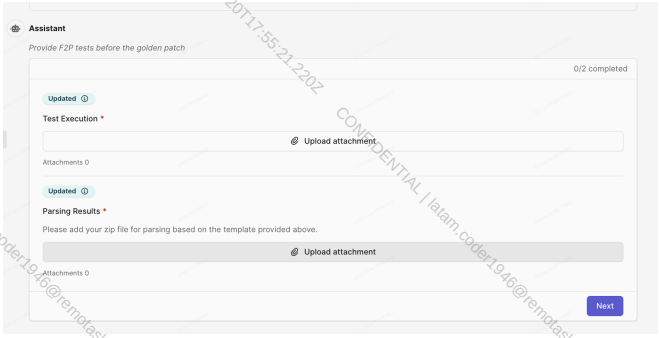
Important for Auditors:

- There are two ways to verify correctness on this project. At least one must be used, and both can be used at the contributor's discretion when they make sense:
    - Atomic rubrics: The contributor's expert rubric must contain a minimum of 5 criteria (likely more) across five dimensions: Instruction Following, Code Correctness, Code Efficiency, Code Clarity, and Code Quality . Rubrics should be used when evaluating outcomes and when we do not want to be prescriptive in the correct implementation.
    - Fail-to-Pass (F2P) Integrity: If F2P tests are used, the automated tests must fail on the buggy/empty codebase and must pass once the Golden Patch is applied. Fail to pass tests must *not* assume an implementation strategy that is not explicitly asked for in the prompt (i.e. if the prompt didn't specify, and the model creates a valid working response, then the tests should pass)
- Environment consistency: All code must execute within the provided Docker container (Ubuntu 22.04) using the standardized `run.sh` and `parsing.py` scripts.

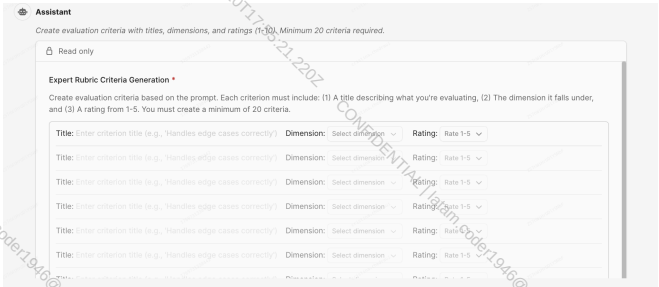
Audit Workflow

1	Review the task instructions: https://app.outlier.ai/en/expert/guidelines/697b72cae052640b8db3e22d

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.220Z

2	Evaluate the Prompt A. This is a rewritten prompt based on the task description B. Rewritten prompts must include an “Expected Interfaces” section which documents publicly accessible/verifier-tested components of the solution (new files, classes, etc, see Expected Interface(s) subdimension for complete description) **02/02**	*Ensure the prompt is a rigorous translation of the natural task description, providing enough context for an LLM to generate the solution and tests. The prompt must contain all constraints that are assumed in the test cases.* 
3	Evaluate the Golden Patch A. Please remember that these tasks are very open-ended, so there might be multiple paths and solutions the contributor may have taken to solve it. We just want to check that the solution makes sense based on the prompt	Verify the **Golden Patch** logic. Check the docker file for technical accuracy. 
4	Evaluate the F2P test cases (if present) A. There should optionally be one set of files before the golden patch intended to fail and after the golden patch intended to pass. B. **02/20** The F2P tests can be verified against the golden patch by running the included validation script `real_coder_e2e.sh` – this script builds the docker image, and runs the test suite before and after adding the codebase to the container. Instructions on running the script are included as comments at the top of the file.	Review the "Before" (Fail) and "After" (Pass) JSON results. Both must be present and deterministic. 

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.220Z

		Please run the validation script below and fix anything that is not passing. **Validation Script:** https://static.remotasks.com/uploads/.../real_c...
5	Evaluate the generated Rubric (if present)	Audit the contributor's **5+ criteria rubric**. Ensure it is atomic and verifiable. 
6	Tally up the final score per the Grading Instructions and make any error category selections (if applicable).	

NOTE: Do not edit tasks or make any selection on the “Task Action” field

General Grading Instructions (How the 1-5 scale is used)	
1	**General Grading**
	Grade to the lowest dimension across all rubrics (e.g. if instruction following is a 2, the task should be rated a 2)
	Grade to the lowest turn for across all turns (e.g. if turn 2 is a 2, the task should be rated a 2)
	If the task meets any criteria under 1-2 Fail, the task is a fail.
	If the task does not fail and it meets criteria for a 3-4 [Not-Fail] on any dimension, then the entire task must be a 3-4 [Not-Fail]
	All dimensions must be a 5 for the task to receive a 5.
2	**Choosing 1 vs 2 or 3 vs 4**
	When deciding between a 1 or 2, select a 1 if the attempter put little to no effort
	When deciding between a 3 or 4, use your best judgement on how serious you think the minor issue affects the quality of the task.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.220Z

3	**Prompt instructions or task instructions should always take precedence over other dimensions**
	For example: if the task instructions asks the user to intentionally make spelling mistakes in the prompt, spelling errors in the prompt would not be marked towards a fail.

Grading Rubrics

Dimension	Sub- Dimension	Notes for Auditors	1 - 2	3 - 4	5
Prompt	**Reasoning Requirement**		**[Fail - Reasoning Requirement Prompt]** - The prompt only requires factual lookup or definition; there is no reasoning at all in the prompt.	**[Non-Fail - Reasoning Requirement Prompt]** - The prompt loosely or trivially includes the skills and/or reasoning of the assigned domain/level of expertise.	- The prompt requires reasoning in-line with the assigned domain/expertise level, beyond just simple recall
Prompt	**Constraints**		**[Fail - Bad Constraints]** - Constraints feel clearly unrealistic or contrived based on the provided context, do not believe that a real user would add these constraints (e.g., sort these books by alphabetical order according to the 2nd letter of the title) - Constraints are clearly unrealistically stacked (e.g., 3 or more formatting constraints)	**- [Fail - Basic Constraints]** - Constraints are pretty basic or may be overused (e.g., keep it brief / I'm in a hurry, explain it to a child, formatting based constraints) - Subjective constraints, where it's believable that somebody in this situation of the prompt may find the constraint useful.	- Any constraints feel natural and are what a real user asking this question to a chatbot would want
Prompt	**Contrived / Unnatural Prompts**		**[Fail - Contrived / Unnatural Prompt]** - Prompt is contrived or otherwise unnatural note: prompts such as riddles that are intentionally constraints/details laden should not be considered contrived.	**[Non-Fail - Somewhat Contrived / Unnatural Prompt]** - Prompt is somewhat contrived or is somewhat unnatural note: prompts such as riddles that are intentionally constraints/details laden should not be considered contrived.	- The prompt is neither contrived or unnatural
Prompt	**Truthfulness**		**[Fail - Major Factual Errors]** - The prompt contains 1 or more major factual errors **[Fail - Minor Factual Errors]** - The prompt contains 2 or more minor factual errors (e.g., statements irrelevant to the fulfillment of the prompt's request)	**[Non-Fail - Minor Factual Errors]** - The prompt contains 1 minor factual errors.	- The prompt has no factual errors - The prompt has no misleading statements
Prompt	**Feasibility**	(03/17) **Note**: It is ok for the prompt to design something that needs to be run with the internet now, but they should still not require usage of external APIs as that would make them unverifiable. 03/21 **NOTE**: Please be aware that there is the possibility of Markdown rendering issues in this prompt section—even if the underlying text is correct, the platform may display it incorrectly. As an example, "duplicates": "\{testlogin\}" may appear as "duplicates": "*testlogin*" due to engineering issues. If something seems	**[Fail - Prompt Impractical Request]** - Prompt contains an impractical request that can't be answered by an LLM in a single response **[Fail - Prompt Impossible Request]** - Prompt has at least one request that can't be fulfilled at all **[Fail - Prompt Conflicting Instructions]** - Prompt gives instructions that conflict/contradict with itself that can't be fulfilled simultaneously	**[Non-Fail - Prompt Impractical Secondary Request]** - There are multiple requests in the prompt and it's verging on being impractical for a model to answer it in a single response, but the core request of the prompt can be fulfilled with minor concessions on the secondary requests	- The prompt is completely actionable by an LLM or chatbot - The prompt contains no conflicting instructions/statements

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.220Z

		suspicious, you can double-check this by looking up the Attempt ID with the Lookup Tool and checking the "Original JSON" section. (Should be under: response.turns[0].prompt.output.content)			
Prompt	**Expected Interfaces** **Added 02/02**	**Expected interface:** CBs must include a section within the rewritten prompt to define every newly introduced file, function, or class that an external application (i.e. not the same codebase) or test suite will interact with. ~~Do not need to flag for helper functions or fields from 3rd party libraries~~ **(03/12)**. It is only about end-to-end functionality, and the overall architectural design of the software. This section must include the below fields: • **Path:** [Exact file path as it appears in the intended structure] • **Name:** [Class.method or function name] • **Type:** [e.g., class, method, function, or interface] • **Input:** [Parameters and types, e.g., chunk: GlibcChunk] • **Output:** [Return type, e.g., None or Promise<void>] • **Description:** [Brief description of observable side effects or behavior asserted by tests] **Language-Specific Fields (As applicable):** • **Inheritance:** extends <Base>; implements <IfaceA, IfaceB> (TypeScript/Java) • **Embedding / Implements:** embeds <TypeA>; implements <IfaceA, IfaceB> (Go) • **Bases / Overrides:** bases: <BaseA, BaseB>; overrides: <Base.method> (Python) • **Annotations / Decorators:** @Override, @Inject, @dataclass, @cached_property, @sealed	**[Fail - Missing Interface Section]** - The expected interface section is missing entirely, or does not include a required field/an applicable language-specific field for any documented interface (see notes). **[Fail - Undocumented Interface]** **02/22: Note**: Do not need to flag for helper functions / trivial import to be included in the expected interface. It is only about end-to-end functionality, and the overall architectural design of the software. - The expected interface section is missing a newly introduced file, function, or class that an external application (i.e. not the same codebase, an external app) or test suite will interact with. Do not reference it back to the golden patch, as the golden patch is only one of the many solutions to the prompt! **[Fail - Misleading Interface Description]** - An interface's documentation is incomplete or insufficient such that an implementation which recreates the requirements fully would still fail a verifier. **[Fail - Invalid Interface] (03/12)** - An interface documented by the CB is a helper function or is a field from a third party library and is not something that an external application would interact with or necessary for it to work	N/A	- The rewritten prompt includes the expected interface section, all required fields within, and any applicable language specific fields **AND** **-** No public interface/interface tested by a verifier is undocumented/documented in a misleading manner.
Golden Response	**Instruction Following / Response Fulfillment**		**[Fail - Explicit Instruction Miss]** - 1 or more explicit instructions are not followed **[Fail - Not Fulfilled]** - The response does not fully answer the question	**[Non-Fail - Subjective Instruction Miss]** - Subjectively misses some aspects of fully answering the question	- All explicit instructions are clearly followed - The response fully answers the question
Golden Response	**Compilation**		**[Fail - Code Compilation Issues]** - The code does not compile	**[Non-Fail - Code Runtime Side Effects]** - The code runs, but has side-effects such as warnings	- Code runs perfectly without any errors or warnings
Golden Response	**Execution Output** **UPDATED 03/16**		**[Fail - Incorrect Code Output]** - Output is irrelevant or incorrect - Output is incomplete - The code throws runtime errors that materially impact the functionality of the golden response, for example an app	**[Non-Fail - Minor Runtime Errors]** - The code has runtime errors that result from a **minor*** edge case, but it does not materially impact the functionality of the golden response other than for this edge case **(03/16)** ***minor =** insignificant enough that it would not need an emergency fix, but would get rolled into the next release in a few weeks for "minor bug fixes"	- Output perfectly aligns with the requirements of the prompt, including edge cases

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.220Z

			crash, unresponsive page, impacted user facing explicitly requested functionality, etc (also see non-fail exception for minor edge cases) **(03/16)**		
Golden Response	**Performance**		**[Fail - Major Code Performance Issues]** - Code implementation is highly inefficient with clear room for significant efficient improvement Ex 1: Takes an O(n^3) brute force approach when it's possible to fulfill the request in O(nlogn)	**[Non-Fail - Minor Code Performance Issues]** - The code implementation is moderately efficient with room for further optimization. Ex: O(n^2) was used when O(nlogn) is possible. This is allowable unless it goes against specific requests from the prompt.	- The code implementation is well-optimized, utilizing efficient algorithms and data structures wherever possible
Golden Response	**Readability**		**[Fail - Major Code Readability Issues]** - The code is difficult to read in > 2 areas because of poor formatting such as missing indentation, poor markdown, excessive white space, or no whitespace (minified code) **[Fail - Misleading Code Variable Names]** - Variable/class/method names are not indicative of their function. Ex: a misleading variable name such as `even_array = [1, 3, 5, 7]`	**[Non- Fail - Minor Code Readability Issues]** - The code can be formatted better in <= 2 areas (because of poor formatting such as missing indentation, poor markdown, excessive white space, or no whitespace (minified code)), but it's still readable - Code has egregious formatting changes but it does not affect the function of the code **[Non-Fail - Poor Code Variable Names]** - Variable/class/method names don't follow the general naming conventions of the respective language	- The code is well organized and uses consistent formatting, making it highly readable - Variable/class/method names are meaningfully chosen and are reflective of their purpose
~~**Golden Response**~~	~~**Code Documentation**~~ **Removed 02/22**	~~Note: Having a properly documented~~ ~~~~README.md~~~~ ~~is ok if the codebase doesn't have excessive comments in every single function~~	~~**[Fail - Misleading Code Documentation]**~~ ~~- Code is very poorly commented (not explaining code in detail) or includes almost no comments~~ ~~Example:~~ ~~# Main function~~ ~~def main():~~ ~~np.random.seed(42)~~ ~~X = np.random.randn(500, 1000)~~ ~~y = np.random.randint(5, size=500)~~ ~~X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)~~ ~~- Code comments are incorrect or redundant.~~	~~**[Non-Fail - Lacking Code Documentation]**~~ ~~- There is a reasonable amount of documentation, but it can be improved to be more detailed~~	~~- All code is well documented and includes detailed comments (Documentation constitutes both code comments and docstrings)~~
Golden Response	**Code Design**		**[Fail - Major Code Design Issues]** - Code implementation uses poor design choices and lacks even the fundamental principles such as modularity and abstraction. It would be hard to maintain and extend this code down the line	**[Non-Fail - Minor Code Design Issues]** - Code implementation partially adheres to design principles (bare minimum) but there is room for improvement	- Code implementation follows most of the design principles, making the code easy to maintain and extend down the line (Code design constitutes good programming practices such as modularity, separation of concerns, abstraction, etc)

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.220Z

Golden Response	**Missing Detail**		**[Fail - Missing Detail]** - The response is overly simplistic and fails to provide a meaningful understanding of the topic. It deals with the topic at a superficial level when a deeper exploration would've been more appropriate **[Fail - Excessive Detail]** - The response is relevant but overly lengthy or complex, totally obscuring key points	**[Non-Fail - Missing Detail]** - The response is generally helpful, but a complete response might need additional detail or nuance **[Non-Fail - Excessive Detail]** - The response is relevant but contains extraneous details that distract from the key points	- The response is clear, focused, and demonstrates a solid understanding of the topic, providing sufficient detail and nuance without overwhelming or distracting from the key points.
Golden Response	**Failing Verifiers** **ADDED 03/25**	**Note:** The Golden Response should pass ALL of the test cases and the rubric criterias that the Contributor writes. If the golden response fails any one of the rubric criteria / unit test, it is an automatic fail.	**[Fail - Golden Response failing verifier]** - The Golden Response fails any of the rubric criteria or any of the F2P tests	N/A	- The golden response is passing all rubric criterias and F2P tests
Test Suite	**Overly Specific/Broad** **Updated 03/25**	**Overly Specific** All the tests in tests.zip should ONLY cover the explicit or implicit backend requirements from the CB's rewritten prompt. They should not be overly specific such that only a subset of solutions pass them, while other valid solutions that fulfill the prompt requirements fail. For example: If the prompt didn't ask for error code handling specifics, do not include error code handling specifics in the unit test, unless it is a strongly implicit requirement of the prompt! **Overly Broad** Overly broad tests are the opposite of overly specific tests--they allow invalid or non-functional implementations to pass. This can include things like static text analysis of generated code instead of testing actual logic, overly permissive assertions, etc. **Note:** If a given request has **both** an appropriately written functional test and one that is overly broad, the second test should not be counted among the total of overly specific/broad tests for this error. Example: - The prompt requests the usage of the pandas library in the project, but the test only verifies that the source code includes a pandas import Also see the examples under Overfitting and Underfitting rubric dimension **(03/12)**	**03/25 [Fail - More Than 10% Overly Specific/Broad Tests]** - More than 10% of the tests are: - Overly specific and test for requirements that do NOT exist in the explicit or implicit requirements of the re-written prompt, - **OR** overly broad when functionality tests are possible in the task scenario (if not possible, do not flag) ~~[Fail - More Than 5% Overly Specific Tests]~~ ~~- More than 5% of the tests are overly specific and test for requirements that do NOT exist in the explicit or implicit requirements of the re-written prompt~~	**03/25 [Non-Fail - Up To 10% Overly Specific/Broad Tests]** - At most 10% of the tests are: - Overly specific and test for requirements that do NOT exist in the explicit or implicit requirements of the re-written prompt, - **OR** overly broad when functionality tests are possible in the task scenario (if not possible, do not flag) ~~[Non-Fail - Up To 5% Overly Specific Tests]~~ ~~- At most 5% (<= 5%) of the tests are overly specific and test for requirements that do NOT exist in the explicit or implicit requirements of the re-written prompt~~	- There are no overly specific unit tests

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.220Z

All CB Generated Content	**Repetitiveness / Relevance**		**[Fail - Major Repetitive Content]** - The content contains unnecessary repetition, having 3 or more sentences that express the exact same idea **[Fail - Not Relevant (4+)]** - The content contains 4 or more irrelevant sentences	**[Non-Fail - Minor Repetitive Content]** - The content contains some unnecessary repetition, having two sentences or phrases that express a similar idea without adding new information. **[Non-Fail - Not Relevant]** - The content contains 3 irrelevant sentences	- The content does not contain unnecessary repetition, having 2 or fewer sentences that express the same idea - The content contains 2 or fewer irrelevant sentences
Core Dimensions	**Clarity Issues**		**[Fail - Clarity Issues]** - The content is extremely difficult to follow or is unclear	**[Non-Fail - Clarity Issues]** - The content makes sense but has some minor clarity issues.	- The response is clear and easy to follow, with well-structured ideas and language that effectively communicates the intended message.
All CB Generated Content	**Unlisted Minor Errors**		N/A	**[Non-Fail - Unlisted Minor Errors]** - There are errors in the task that are not explicitly mentioned in the grading rubrics but prevent the task from being perfect.	- There are no unlisted errors that you feel degrade the quality of the task
All CB Generated Content	**Original Work**		**[Fail - LLM Cheating]** - The content contains clear and obvious evidence of unsanctioned LLM usage in their work **[Fail - Plagiarism]** - The content contains direct plagiarism without citation Note: Evidence of chatbot usage could be excessive usage of fluff / pleasantries, generic responses, lack of human understanding and nuance, etc. Copy paste prompts that ask for summaries or analysis of the pasted material are okay.	N/A	**-[Non-Fail - Suspected LLM Cheating]** - The content shows evidence of unsanctioned LLM usage or otherwise appears unnatural, but it is not obvious that an LLM was used to generate their work - The content does not contain clear or obvious evidence of LLM usage - The content does not contain direct plagiarism without citation
All CB Generated Content	**Harmful Content**		**[Fail - Harmful Content]** - The material contains any harmful content and the project does not call for it	N/A	- Content does not contain or ask about harmful content
Data Specification Completeness	**Data Specification Completeness**	Critical Fields: - Dockerfile ~~- build script (build.sh)~~ **(removed 02/19)** - golden patch - At least one verification method (rubric and/or test suite) Additionally, if a test suite is included: - test execution script (run.sh) - test output parsing script (parsing.py) - test execution results (before & after applying the patch) - f2p test list	**[Fail - Missing Critical Fields]** - Any critical field (see notes) is missing.	N/A	All dimensions (Task ID, Prompt, Docker URL, Eval Script, Rubrics, Reference Answer, etc.) are defined with clear data types and detailed descriptions of their role.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.220Z

Description	**Invalid Descriptions**		**[Fail - Invalid Description]** - The rewritten prompt doesn't match the task description, and fundamentally alters the overall context to create a significantly different request.	**[Non-Fail - Misaligned Description]** - The rewritten prompt includes some minor departures from the task description, but does not fundamentally alter the context or goal of the original request.	
Environments	**Environment Compatibility**		**[Fail - Incompatible Environment]** - The test suite (if present) requires external databases or APIs that aren't provided in the container.	N/A	The test suite is "portable"; it can be run immediately within the Docker environment with zero manual configuration

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.220Z

Verifiers	**Verifier Coverage**	Tests (if present) and rubric criteria when considered together should verify all the requests in the rewritten prompt. **(03/04) Note**: Do not flag for coverage on any subjective UI Design requirements (e.g. elegant, pretty, good, any design style) since it is not the customer's focus. Do not flag any trivial / minor UI design requirements out of the prompt that the contributor writes, since this queue is not UI Design focused, it is code focused. For example: - Prompt: "... add navigation to the left menu..." - QC should NOT flag for missing coverage if the verifiers don't check if the code puts the menu on the left. QC should still check to see if there are criteria that verify the menu code's functionality, though. **(02/27) Note:** A section of the expected interface is considered "covered" if it contains at least one representative unit test that, upon failure, would indicate a breakdown of all related logic in that section of the expected interface. You do not need to flag for missing coverage on minor details if this "bottleneck" test is present. **02/22** Note: Do not need to check if the golden patch's backend are all tested by the test suite. You should only check if the test suite is checking all the explicit backend requirements from the prompt only. Only penalize if the test suite is not checking any backend requirements. It is also okay if the tests are generic as it's meant to handle multiple possible solutions. **Note 02/02:** Prompt instructions **explicitly stated to be optional** may be omitted from the verifiers, however, ensure that it is indeed the **instruction** that is optional (i.e., that the prompt is making a mandatory request concerning a feature that is optional in some other context, e.g., from the perspective of a user or customer) Example 1 (should not be failed): - Prompt: "...You can also include a light mode/dark mode toggle" - Rubric/tests make no mention of this toggle, and this is ok. Example 2 (should be failed) - Prompt: "...the customer can optionally include a note with	**[Fail - Major Insufficient Verifier Coverage]** - The test suite is missing coverage for more than 5% of **major** backend requirements of the prompt (not the golden patch), and the rubric also does not cover it at all. See notes **(03/17)**	**[Non-Fail - Minor Insufficient Verifier Coverage]** - The test suite is missing at least one test which checks for a non-critical implicit requirement of the prompt, and this requirement is not verified by any rubric criterion. - The test suite is missing coverage for up to 5% of **major** backend requirements of the prompt (not the golden patch), and the rubric also does not cover it at all. See notes **(03/17)**	- The verifiers (rubric criteria and test suite), in combination, address all explicit and implicit requirements of the rewritten prompt.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.220Z

		each order" - The rubric/tests must verify that the ability to add this note is present in the response. If the coverage for one test is sufficiently accounted for by other tests ***collectively***, then that test is not necessary but can be added **(03/17)** Example: ● Test 1 checks that the first element in a list is 1 ● Test 2 checks that the second element in a list is 2 ● We don't need a test 3 checking that the list is [1,2] (assuming it's a 2 item list)			
Rubrics	**Overall Rubric Quality** **ADDED 02/19**	Issues with rubric criteria are divided into three types - Major, Moderate and Minor. All the errors across rubrics are tallied at the end to arrive at a holistic rating for the entire rubric. See the next columns to understand the thresholds. **See Rubric Quality Definitions section below to understand what counts as a Major, Moderate and Minor issue.** Use the number of criteria that the CB wrote as the denominator while calculating % values. Numerator is the number **of criteria with issues of the type** (see definition below) among criteria. Take note that some issues have bespoke counting rules (mentioned under "Definition"). **Do NOT double count criteria while tallying even if it has multiple issues. If a criteria has 1 major issue and 1 moderate issue, count it as having 1 major issue only.** **03/04 Note: CHECK THE "WHEN TO FLAG" DEFINITION FOR "MISSING CRITERIA" TO MAKE SURE YOU'RE FLAGGING CORRECTLY.**	**Use the number of criteria that the CB wrote as the denominator while calculating % values. See the additional notes section for the numerator. Do NOT double count criteria while tallying even if it has multiple issues.** **[Fail - 5%+ Major Rubric Errors]** - More than 5% of the criteria contain major issues **[Fail - 15%+ Moderate Rubric Errors]** - More than 15% of the criteria contain moderate or major issues **[Fail - 25%+ Minor Rubric Errors]** - More than 25% of the criteria contain minor or moderate or major issues	**Use the number of criteria that the CB wrote as the denominator while calculating % values. See the additional notes section for the numerator. Do NOT double count criteria while tallying even if it has multiple issues.** **[Non-Fail - Up to 5% Major Errors]** - Up to 5% (<=5%) of the criteria contain major issues **[Non-Fail - Up to 15% Moderate Errors]** - Up to 15% (<=15%) of criteria contain moderate or major issues (with major issues contributing lower than 5%) **[Non-Fail - 5-25% Minor Errors]** - Between 5 and 25% (>=5% and <=25%) of criteria contain minor or moderate or major issues (with major issues contributing lower than 5% and moderate issues contributing lower than 15%)	- Less than 5% (<5%) of the rubrics have minor issues - No major or moderate issues

Appendix

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.220Z

Rubric Quality Definitions (02/19)

Major Issues	**Missing Criteria - Critical Requirements**	**Definition:** • Count each missing rubric **(*see notes below)** that ought to check for an explicit requirement in the prompt or a critical implicit expectation of the prompt as one issue (critical = you cannot imagine a good response without it) We only need to cover the backend requirement with F2P tests, frontend requirements by rubrics, and anything that is not coverable by F2P tests to be covered by Rubrics. We should NOT penalize the coverage of rubrics for full coverage on all requirements of the re-written prompt as long as the F2P test covers it. **02/21:** To avoid requiring overly lengthy CB written rubrics, we only need to cover the max 30 most important things that cannot be covered by unit tests AND can be covered by rubrics only, while maintaining the atomicity rule above. Do not penalize for coverage on minor requirements. Note: This batch is permitted to have more coverage on minor requirements That is to say, If a prompt has 45 explicit requirements, if 30 have criteria coverage, we don't need to flag the remaining 15. If a prompt has 20 explicit requirements, ALL 20 should have criteria. When to Flag or Not Flag for **Missing Critical Criteria**: • If there are <30 explicit request criteria: any criteria missing should be flagged only if there are *more than 3* important explicit requested criteria missing **(02/22)** • If there are >= 30 explicit request criteria: any criteria missing should **not** be flagged UNLESS *more than 3* important explicit requested criteria that should be top 30 are missing AND the rubric covers non-critical criteria instead of the explicit requests. **(02/22)** 02/22 NOTE: This error category only applies to requirements that unit tests didn't touch.
	Criteria Not Self Contained	**Definition:** Criterion cannot be evaluated against the model response without access to the prompt, reference text, other criteria, and/or external facts/information Every rubric must be self-contained. Imagine that you only have access to the model response and are trying to evaluate if it fulfilled the rubric item. Will you be able to evaluate accurately without referencing anything else? Some criteria are self-explanatory but often, this translates to the criteria mentioning the answer to the prompt directly. Examples of criteria that are not self-contained: • Example 1: "Response identifies the first president of the USA" ○ Fixed: "Response identifies the first president of the USA as George Washington" • Example 2: "The response addresses the bug mentioned in the prompt"

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.220Z

		○ Fixed: "The response addresses the bug where the submit button doesn't work"
	Criteria Not Atomic - Major **UPDATED 02/26**	**Definition:** Criterion groups two or more constraints that are **completely unrelated**, which results in a rubric item with no clear focus on what aspect of the response it's trying to evaluate. These constraints cannot be interpreted as part of a single coherent instruction but reads more like a dump of requirements. Given the broadness of the task, it is acceptable (i.e., do not flag these, even as minor) to have a "higher level" rubric that combines constraints as one coherent instruction, such as "implements a tech stack that includes x, y, z" (where x, y, z were specified in the prompt). The purpose of this is to avoid a proliferation of rubrics that test the same concept and either under or over penalizing the model. To qualify as a major failure, a criterion must bundle totally unrelated constraints. Similarly, features that are closely related can also be grouped into one criterion and QC should not flag this as an atomicity issue. This is so that there is as much coverage on all the features requested in the prompt as possible (there can be a lot of features!) **(02/26)**
	Incorrect Criteria	**Definition:** • Criterion checks for something that does not align with prompt requirements • Criterion contains a factual error or a misleading point ○ Example: "The response implements a sorting algorithm that runs in O(nlogn), such as selection sort" • Criterion is not an explicit requirement in the prompt and implementing it does not make the response worse • Criterion is not at all related to the requests in the prompt **NOTE:** Before classifying any issue as "Incorrect criteria", see if a different, more specific error category would apply. For example, if a criterion is overly specific, you could argue that it's "incorrect", but it should still be counted as "Overfitting and Underfitting".
	Framing	**Definition:** • Criteria that are not framed in a way such that a good response would evaluate to "Yes" or "True" Criteria should be positively *framed*. Meaning, they should all evaluate to "True" or "Yes" or "Pass" for a good response. Incorrectly framed criteria are as bad as incorrect criteria.
Moderate Issues	**Missing Criteria — Non-critical Requirements**	**Definition:** • Count each missing rubric that ought to check for a non-critical explicit requirement or implicit expectation of the prompt as one issue (critical = you cannot imagine a good response without it) • Example non-critical requirements: "Use bold text", "Use bullet points" We only need to cover the backend requirement with F2P tests, frontend requirements by rubrics, and anything that is not coverable by F2P tests to be covered by Rubrics. We should NOT penalize the coverage of rubrics for full coverage on all requirements of the re-written prompt as long as the F2P test covers it. **02/21**: To avoid requiring overly lengthy CB written rubrics, we only need to cover the top 30 most important things that can be covered

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.220Z

		by rubrics only, while maintaining the atomicity rule above. Do not penalize for coverage on minor requirements. Do not penalize for having more than 30 rubrics for this batch since this batch is permitted to have more coverage on minor requirements.
	Overlapping or Redundant Criteria	**Definition:** • Criterion that is either completely redundant because other criteria completely encompass the former or multiple criteria that check for the same thing partly • Count each completely redundant criteria as one moderate issue or count multiple overlapping criteria as one moderate issue Redundant Scenario: Criteria 1: Response does a, b, c Criteria 2: Response does a, b Overlap Scenario: Criteria 1: Response does a, b, c Criteria 2: Response does b, c, d Note that this applies to cases where two criteria independently assess the same elements, not when a single criterion introduces and specifies related requirements. ("The response follows best code practices by ensuring that each line is under 79 characters" is acceptable.)
	Overfitting and Underfitting **UPDATED 03/12**	**Definition:** • **Overfitting:** Criteria that are overly specific, inflexible or too rigid - they correctly accept some valid implementations but also incorrectly reject a subset of valid implementations • **Underfitting:** Criteria that are overly broad, permissive or loose - they accept valid implementations, but also incorrectly accept invalid implementations too Criteria must be flexible enough to accept different valid implementations and only valid implementations. Note that criteria can mention specific answers as long as they are provided as examples in any way. i.e. within parentheses, or along with "for example" wording, or any other form which does not limit the answer. **Overfitted vs Nice-to-have criteria:** • If a criterion could be considered necessary for a perfect response but it wasn't explicitly required, it should not be considered an overfitted criterion. Rather, these are considered Nice-To-Have (i.e., assigned a weight of 1) • One example of this is, if a prompt mentions that a feature is "optional but recommended", criteria checking for this optional feature must be written, but given a weight of 1 **Overfit Examples:** • If the prompt didn't specify that a perfect response needs to use any specific document name / file name / file path but the rubric is flagging any solution for not using a specific file path / spelling (that is used in golden patch only for example) • Provide a list of films, along with their release years as a CSV file "Example headers: Film, Year"

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.220Z

		○ Criterion checks that the headers are named “Film” and “Year” ○ This is overly specific because the prompt used those names as an example. Alternative header names such as “Movie” and “Release Year” would be unfairly rejected.
	Subjective Criteria	**Definition:** • Criteria that are subjective, vague or immeasurable (e.g., “the response should have good formatting” or “code must be optimal”) A criterion should be evaluated based on whether its primary requirement is measurable, even if it includes additional context or reasoning that's less precise Using vague or subjective qualifiers like “appropriate”, “properly”, “best practices”, “reasonable” etc without attaching explicit definitions makes criteria unmeasurable and should be flagged here. Subjectivity with certain details should be acceptable when the prompt is intentionally ambiguous and open-ended For example, for a prompt "Create an artistic website to showcase my sculptures, featuring an animated background that looks like shifting, fluid marble", the following rubric is acceptable: "The website has a refined modern look so it could be launched as a product by a reputable company."
	Incorrect Weights - Major	**Definition:** • Criteria that are objectively incorrectly weighted by two levels, i.e., 1 is selected when 5 is appropriate or vice versa Criteria are categorized into one of 3 weight buckets: 1 (low), 3 (medium), or 5 (high)
	~~**Criteria Not Atomic - Minor**~~ **REMOVED 02/20**	~~**Definition:**~~ ~~• Criterion groups two or more constraints that are only partially related such as ... NOTHING!~~
Minor Issues	**Incorrect Weights - Minor**	**Definition:** • Criteria that are objectively incorrectly weighted by one level. i.e., 1v3 or 3v5 scenarios Criteria are categorized into one of 3 weight buckets: 1 (nice-to-have), 3 (important), or 5 (critical)
	Miscategorized Criteria	**Definition:** Criteria are objectively tagged with the wrong category when there is a better one available. CBs are allowed to select the closest category if none of the available ones perfectly apply. **NOTE:** There are a lot of cross-matches between Instruction Following and the categories due to the expected level of details in the prompt. If the CB assigned one of the Code Categories instead of IF, this should not be flagged as long as the Code category assignment is also applicable. In general, be on the lookout for overlapping categories, as long as one of the applicable categories is selected, that’s fine! **(03/23)** **List of categories:**

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.220Z

		• Instruction Following: Ensures the response adheres to explicit directions in the prompt (format, constraints, language, libraries, required elements). • Code Correctness: Ensures any code performs the intended task and produces correct results based on the prompt. • Code Quality: Covers robustness, maintainability, idiomatic patterns, and avoiding fragile or error-prone design choices. • Code Clarity: Covers readable, well-structured code, including naming, organization, and formatting. • Code Efficiency: Covers conciseness, avoidance of unnecessary steps, and reduction of redundancy.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.220Z

Examples

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.220Z

Use this space to list any attempts that are good examples of the application of a rubric principle

Consider using the test audits from the Rubric Generation Phase OR notable disputes

Attempt_id	Relevant Rubric dimension	Summary of issues	Correct Application of standard

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.220Z